

Complete In-Depth Notes on JavaScript Promises

Table of Contents

1. Introduction to Promises
 2. Understanding Asynchronous Operations
 3. Promise States
 4. Creating Promises
 5. Consuming Promises
 6. Promise Chaining
 7. Error Handling with Promises
 8. JSON vs JavaScript Objects
 9. Practical Implementation
 10. Advanced Concepts
-

1. Introduction to Promises

What is a Promise?

Official Definition:

"The Promise object represents the eventual completion (or failure) of an asynchronous operation and its resulting value."

Simplified Explanation: A Promise is a special JavaScript object that handles asynchronous operations. Think of it as a "promise" that JavaScript makes to you - it promises to eventually give you an answer, but that answer might take some time to arrive.

Why Do We Need Promises?

In the previous lecture on Callback Hell, we saw how nested callbacks made code:

- Difficult to read
- Hard to maintain
- Confusing to debug
- Created "pyramid of doom" structure

Promises solve these problems by:

- Making asynchronous code more readable
 - Providing better error handling
 - Avoiding deeply nested callbacks
 - Giving us control over execution flow
-

2. Understanding Asynchronous Operations

Real-World Example: Fetching GitHub User Data

Let's understand promises through a practical example of building a webpage that displays GitHub user information.

Requirements:

- Display photos of 30 GitHub users
- Show their usernames
- Provide links to their profiles

The Problem: To display this information, we first need to **fetch** the data from GitHub's API. This is an asynchronous operation because:

1. Our client (browser) sends a request to GitHub's server
2. GitHub's server processes the request
3. The server sends back the data
4. This entire process takes time (could be 100ms, 1 second, or longer)

Key Point: JavaScript won't wait for this data. It continues executing the next lines of code immediately.

The Fetch Operation

```
// Fetching data from GitHub API
const p1 = fetch("https://api.github.com/users");
console.log(p1); // Output: Promise { <pending> }
```

What happens here:

1. **Request is sent** to GitHub's API
2. **Immediately returns** a Promise object
3. The Promise is in **"pending"** state
4. JavaScript moves to the next line without waiting

3. Promise States

A Promise can be in one of **three states**:

3.1 Pending State

CLIENT ----[Request]----> GITHUB SERVER

↓

P1 = Promise { <pending> }

- **When:** Right when you make the request
- **Meaning:** The operation is in progress
- **What to do:** Don't try to read the data yet - there's nothing there!

3.2 Fulfilled State (Resolved)

CLIENT <---[Data Received]--- GITHUB SERVER

↓

P1 = Promise { <fulfilled>: data }

- **When:** The server successfully responds with data
- **Meaning:** You got what you asked for
- **What to do:** Now you can read and use the data

3.3 Rejected State

CLIENT <---[Error/Failure]--- GITHUB SERVER

↓

P1 = Promise { <rejected>: reason }

- **When:** The operation failed
- **Meaning:** Something went wrong
- **What to do:** Handle the error appropriately

Understanding Rejection vs Fulfillment

CRITICAL CONCEPT: Many developers misunderstand when rejection happens.

Rejection occurs ONLY when:

1. Your internet is down

2. The server is down/unreachable
3. DNS server cannot resolve the URL

Fulfillment occurs when:

1. Server responds with data ✓
2. Server responds with "data not found" ✓
3. Server responds with "bad request" ✓
4. Server responds with any status code ✓

Key Point: If the server responds (even with an error message), the Promise is **fulfilled**, not rejected. It's YOUR responsibility to check if the response contains actual data or an error.

4. Creating Promises

Basic Syntax

```
const p1 = new Promise((resolve, reject) => {  
  // Your asynchronous operation here  
  
  if (/* operation successful */) {  
    resolve(data); // Fulfill the promise  
  } else {  
    reject(error); // Reject the promise  
  }  
});
```

Example 1: Simple Promise

```
const p1 = new Promise((resolve, reject) => {  
  resolve("Hello");  
});  
  
console.log(p1); // Promise { <fulfilled>: "Hello" }
```

Example 2: Promise with Object Data

```
const p1 = new Promise((resolve, reject) => {  
  resolve({  
    name: "Rohit",  
    age: 30  
  });  
});
```

```
});
```

```
console.log(p1); // Promise { <fulfilled>: { name: "Rohit", age: 30 } }
```

Example 3: Rejected Promise

```
const p1 = new Promise((resolve, reject) => {  
  reject("Error occurred");  
});
```

```
console.log(p1); // Promise { <rejected>: "Error occurred" }  
// This will cause an unhandled promise rejection error!
```

Example 4: Promise with Timeout

```
const placeOrder = (orderDetails) => {  
  return new Promise((resolve, reject) => {  
    setTimeout(() => {  
      console.log("Payment is in progress");  
      orderDetails.address = true;  
      resolve(orderDetails);  
    }, 3000);  
  });  
};
```

Explanation:

- Function returns a Promise
- Simulates async operation with `setTimeout`
- After 3 seconds, modifies `orderDetails`
- Resolves with updated data

5. Consuming Promises

The `.then()` Method

The `.then()` method is used to handle fulfilled promises.

Syntax:

```
promise.then((data) => {  
  // Handle the data  
});
```

Example: Consuming Fetch Promise

```
const p1 = fetch("https://api.github.com/users");
```

```
p1.then((response) => {  
  console.log(response);  
});
```

Output:

```
Response {  
  status: 200,  
  statusText: "OK",  
  body: ReadableStream,  
  // ... other properties  
}
```

Understanding the Response Object

When you fetch data, you get a Response object, not the actual data immediately. The data is in a special format (bytes) that needs to be converted.

```
p1.then((response) => {  
  console.log(response.body); // ReadableStream  
  // Data is in byte format, not readable yet!  
});
```

Converting Response to JSON

```
p1.then((response) => {  
  return response.json(); // Returns another Promise!  
});
```

Important: `response.json()` is also asynchronous! It returns another Promise because:

- Converting bytes to JavaScript objects involves heavy computation
- JavaScript doesn't want to block execution

- It's treated as an async operation
-

6. Promise Chaining

Promise chaining allows us to perform sequential asynchronous operations in a clean, readable way.

Problem Without Chaining

```
const p1 = fetch("https://api.github.com/users");

p1.then((response) => {
  const p2 = response.json();

  p2.then((data) => {
    console.log(data);
  });
});
```

This works but is still somewhat nested. We can do better!

Solution: Promise Chaining

```
fetch("https://api.github.com/users")
  .then((response) => {
    return response.json();
  })
  .then((data) => {
    console.log(data);
  });
```

How it works:

1. `fetch()` returns a Promise → P1
2. When P1 resolves, first `.then()` executes
3. First `.then()` returns `response.json()` (another Promise) → P2
4. When P2 resolves, second `.then()` executes
5. Second `.then()` receives the actual data

Visual Flow

fetch()
↓
Promise (pending)
↓
.then(response) - receives Response object
↓
return response.json()
↓
Promise (pending)
↓
.then(data) - receives actual data
↓
console.log(data)

Complete Working Example

```
fetch("https://api.github.com/users")
  .then((response) => {
    return response.json();
  })
  .then((data) => {
    // Display user photos
    const parent = document.getElementById("first");

    for(let i = 0; i < data.length; i++) {
      const img = document.createElement("img");
      img.src = data[i].avatar_url;
      img.style.width = "40px";
      img.style.height = "40px";
      parent.appendChild(img);
    }
  });
```

What this does:

1. Fetches data from GitHub API
 2. Converts response to JavaScript objects
 3. Loops through all 30 users
 4. Creates an image element for each user
 5. Sets the image source to user's avatar URL
 6. Appends image to the page
-

7. Error Handling with Promises

The .catch() Method

The `.catch()` method handles rejected promises.

Syntax:

```
promise
  .then((data) => {
    // Handle success
  })
  .catch((error) => {
    // Handle error
  });
```

When Does .catch() Execute?

ONLY in these three scenarios:

Internet is down

```
// User's internet connection is lost
fetch("https://api.github.com/users")
  .catch((error) => {
    console.log("No internet connection");
  });
```

1.

Server is down

```
// GitHub's server is unreachable
fetch("https://api.github.com/users")
  .catch((error) => {
    console.log("Server is down");
  });
```

2.

DNS cannot resolve URL

```
// DNS server cannot convert URL to IP address
fetch("https://api.github.com/users")
```

```
.catch((error) => {  
  console.log("DNS resolution failed");  
});
```

3.

Handling Server Errors (Not Caught by .catch())

Common Misunderstanding: If the URL is wrong or data doesn't exist, `.catch()` won't execute because the server RESPONDED (even if with an error).

Example:

```
fetch("https://api.github.com/usersssss") // Wrong URL  
  .then((response) => {  
    console.log(response.ok); // false  
    console.log(response.status); // 404  
    // This still executes! Server responded with 404  
  })  
  .catch((error) => {  
    // This does NOT execute!  
    // Server responded, so promise is fulfilled  
  });
```

Proper Error Handling

```
fetch("https://api.github.com/users")  
  .then((response) => {  
    // Check if response is successful  
    if (!response.ok) {  
      throw new Error("Data is not present in server");  
    }  
    return response.json();  
  })  
  .then((data) => {  
    // Handle successful data  
    console.log(data);  
  })  
  .catch((error) => {  
    // Now catches both:  
    // 1. Network errors  
    // 2. Errors we manually threw  
    console.log("Error:", error);  
  });
```

Complete Error Handling Example

```
fetch("https://api.github.com/users")
  .then((response) => {
    console.log(response);

    if (!response.ok) {
      throw new Error("Data is not present in server");
    }

    return response.json();
  })
  .then((data) => {
    const parent = document.getElementById("first");

    for(let i = 0; i < data.length; i++) {
      const img = document.createElement("img");
      img.src = data[i].avatar_url;
      img.style.width = "40px";
      img.style.height = "40px";
      parent.appendChild(img);
    }
  })
  .catch((error) => {
    const parent = document.getElementById("first");
    parent.textContent = error.message;
  });
```

This handles:

- Network failures
- Server errors
- Missing data
- Invalid URLs

8. JSON vs JavaScript Objects

Common Confusion

Many developers think JSON means "JavaScript Object". This is **wrong!**

JSON stands for: JavaScript Object Notation

Key Difference:

- **JSON** = Universal data format (understood by ALL programming languages)
- **JavaScript Object** = Only understood by JavaScript

Why JSON Exists

Problem: Different programming languages need to communicate

Client (JavaScript) $\longleftrightarrow$ Server (Python/C++/Java/etc.)

Solution: A common format that everyone understands = JSON

JSON vs JavaScript Object - Detailed Comparison

Feature	JavaScript Object	JSON
Keys	Can be without quotes	MUST be in double quotes
Values	Strings can use single or double quotes	MUST use double quotes
undefined	Allowed	NOT allowed
Functions	Allowed	NOT allowed
Trailing comma	Allowed	NOT allowed
Format	JavaScript data structure	String format
Language Support	Only JavaScript	All languages

Examples

JavaScript Object:

```
const js1 = {  
  name: "Rohit",    // No quotes on key - OK  
  age: 30,  
  address: 'Dwarka', // Single quotes - OK  
  c: undefined,     // undefined - OK  
};
```

JSON Format:

```
const jsonFormat = '{"name":"Rohit","age":30,"address":"Dwarka"}';  
// Everything in double quotes  
// No undefined  
// No trailing comma  
// Entire thing is a STRING
```

Converting Between JSON and JavaScript

JavaScript Object → JSON

```
const js1 = {  
  name: "Rohit",  
  age: 30,  
  address: "Dwarka",  
  c: undefined  
};  
  
const jsonFormat = JSON.stringify(js1);  
  
console.log(jsonFormat);  
// Output: '{"name":"Rohit","age":30,"address":"Dwarka"}'  
// Note: undefined was automatically removed!
```

What **JSON.stringify()** does:

- Converts JavaScript object to JSON string
- Removes **undefined** values
- Ensures all keys are in double quotes
- Removes trailing commas
- Makes it sendable over the network

JSON → JavaScript Object

```
const jsonFormat = '{"name":"Rohit","age":30,"address":"Dwarka"}';  
  
const jsObject = JSON.parse(jsonFormat);  
  
console.log(jsObject);  
// Output: { name: 'Rohit', age: 30, address: 'Dwarka' }  
// Now it's a real JavaScript object!
```

What **JSON.parse()** does:

- Converts JSON string to JavaScript object
- Makes it usable in JavaScript code
- Allows you to access properties with dot notation

Why JSON is a String

JSON is stored as a string because:

1. **Network transmission** - Data travels as bytes over the internet
2. **Character encoding** - Each character has a corresponding byte value
3. **Universal compatibility** - Strings can be easily converted to bytes

JSON String → Bytes → Network → Bytes → JSON String → Parse → Object

9. Practical Implementation

Solving the Callback Hell Problem

Remember the callback hell from the previous lesson? Let's solve it with Promises!

Original Callback Hell Code

```
const orderDetails = {
  items: "Pizza, Biryani, Coke",
  total: 620
};

placeOrder(orderDetails, () => {
  prepareOrder(orderDetails, () => {
    pickupOrder(orderDetails, () => {
      deliverOrder(orderDetails, () => {
        console.log("Order delivered!");
      });
    });
  });
});
```

Problems:

- Deeply nested

- Hard to read
- Difficult to maintain
- Inversion of control

Solution with Promises

```
const orderDetails = {
  items: "Pizza, Biryani, Coke",
  total: 620
};

placeOrder(orderDetails)
  .then((orderDetails) => {
    return prepareOrder(orderDetails);
  })
  .then((orderDetails) => {
    return pickupOrder(orderDetails);
  })
  .then((orderDetails) => {
    return deliverOrder(orderDetails);
  })
  .then((orderDetails) => {
    console.log(orderDetails);
  })
  .catch((error) => {
    console.log("Error:", error);
  });
```

Benefits:

- Flat structure
- Easy to read
- Clear flow
- Centralized error handling
- We maintain control

Implementing the Functions

1. Place Order Function

```
const placeOrder = (orderDetails) => {
  return new Promise((resolve, reject) => {
    setTimeout(() => {
      console.log("620 Payment is in progress");
    }, 2000);
  });
};
```

```

    // Simulate payment success/failure
    if (Math.random() > 0.1) { // 90% success rate
      console.log("Payment received and order is placed");
      orderDetails.address = true;
      resolve(orderDetails);
    } else {
      reject("Payment failed");
    }
  }, 3000);
});
};

```

Explanation:

- Returns a Promise
- Simulates 3-second payment process
- 90% chance of success
- Adds `address: true` to orderDetails on success
- Resolves with updated orderDetails
- Rejects if payment fails

2. Prepare Order Function

```

const prepareOrder = (orderDetails) => {
  return new Promise((resolve, reject) => {
    setTimeout(() => {
      console.log("Your food preparation started");

      // Simulate preparation success/failure
      if (Math.random() > 0.05) { // 95% success rate
        console.log("Your order is now prepared");
        orderDetails.tokenId = 123;
        resolve(orderDetails);
      } else {
        reject("Food item is not present at restaurant");
      }
    }, 3000);
  });
};

```

Explanation:

- Returns a Promise
- Simulates 3-second preparation

- 95% success rate
- Adds `tokenId: 123` for pickup
- Resolves with updated `orderDetails`

3. Pickup Order Function

```
const pickupOrder = (orderDetails) => {
  return new Promise((resolve, reject) => {
    setTimeout(() => {
      console.log("Delivery boy is on the way to pick up order");

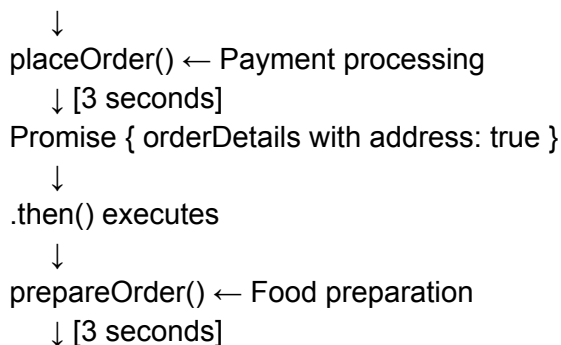
      if (Math.random() > 0.05) { // 95% success rate
        console.log("I have picked up the order");
        orderDetails.received = true;
        resolve(orderDetails);
      } else {
        reject("Unable to reach restaurant");
      }
    }, 3000);
  });
};
```

4. Deliver Order Function

```
const deliverOrder = (orderDetails) => {
  return new Promise((resolve, reject) => {
    setTimeout(() => {
      console.log("I am on the way to deliver through Dwarka");
      resolve(orderDetails);
    }, 3000);
  });
};
```

Complete Flow Visualization

USER INITIATES ORDER



Promise { orderDetails with tokenId: 123 }
↓
.then() executes
↓
pickupOrder() ← Delivery boy picks up
↓ [3 seconds]
Promise { orderDetails with received: true }
↓
.then() executes
↓
deliverOrder() ← Delivery in progress
↓ [3 seconds]
Promise { complete orderDetails }
↓
.then() executes
↓
console.log(orderDetails)
↓
FINALLY

Error Flow

ANY STEP FAILS

↓
reject() is called
↓
Skip all remaining .then()
↓
Jump directly to .catch()
↓
console.log("Error:", error)
↓
FINALLY (still executes!)

10. Advanced Concepts

The .finally() Method

The `.finally()` method executes regardless of whether the Promise was fulfilled or rejected.

Syntax:

```
promise
  .then((data) => {
    // Handle success
  })
  .catch((error) => {
    // Handle error
  })
  .finally(() => {
    // Always executes
  });
```

Use Cases for .finally()

1. Hiding Loading Spinner

```
// Show loader
showLoader();
```

```
fetch("https://api.github.com/users")
  .then((response) => {
    return response.json();
  })
  .then((data) => {
    displayData(data);
  })
  .catch((error) => {
    showError(error);
  })
  .finally(() => {
    // Hide loader regardless of success/failure
    hideLoader();
  });
```

Why this is useful:

- User sees loading indicator during fetch
- Whether data loads successfully or fails, loader is removed
- No need to duplicate `hideLoader()` in both `.then()` and `.catch()`

2. Database Connection Cleanup

```
connectToDatabase()
  .then((connection) => {
    return queryData(connection);
```

```

})
.then((data) => {
  processData(data);
})
.catch((error) => {
  handleError(error);
})
.finally(() => {
  // Close database connection
  // This happens whether query succeeded or failed
  closeConnection();
});

```

3. Cleanup Operations

```

openFile()
  .then((file) => {
    return readFile(file);
  })
  .then((content) => {
    processContent(content);
  })
  .catch((error) => {
    logError(error);
  })
  .finally(() => {
    // Always close the file
    closeFile();
  });

```

Complete Example with .finally()

```

placeOrder(orderDetails)
  .then((orderDetails) => {
    return prepareOrder(orderDetails);
  })
  .then((orderDetails) => {
    return pickupOrder(orderDetails);
  })
  .then((orderDetails) => {
    return deliverOrder(orderDetails);
  })
  .then((orderDetails) => {
    console.log(orderDetails);
  });

```

```
})  
.catch((error) => {  
  console.log("Error:", error);  
})  
.finally(() => {  
  console.log("I am doing cleanup");  
  // This executes whether order succeeded or failed  
});
```

Output (Success Case):

620 Payment is in progress
Payment received and order is placed
Your food preparation started
Your order is now prepared
Delivery boy is on the way to pick up order
I have picked up the order
I am on the way to deliver through Dwarka
{ items: 'Pizza, Biryani, Coke', total: 620, address: true, tokenId: 123, received: true }
I am doing cleanup

Output (Failure Case):

620 Payment is in progress
Error: Payment failed
I am doing cleanup

Browser Storage Restriction

CRITICAL: In Claude.ai artifacts, you CANNOT use:

- `localStorage`
- `sessionStorage`
- Any browser storage APIs

Instead use:

- React state (`useState`, `useReducer`)
- JavaScript variables
- In-memory storage

Why? These APIs are not supported in the Claude.ai environment and will cause your artifact to fail.

Key Takeaways

1. Promise States

- **Pending:** Operation in progress
- **Fulfilled:** Operation completed successfully
- **Rejected:** Operation failed (ONLY for network/server issues)

2. When Rejection Occurs

Only these three cases:

1. Internet is down
2. Server is down
3. DNS cannot resolve URL

Everything else (404, 500, bad request) is **fulfilled!**

3. Promise Methods

- `.then()` - Handle fulfilled promises
- `.catch()` - Handle rejected promises
- `.finally()` - Always executes (cleanup)

4. JSON vs JavaScript Object

- JSON = String format, universal
- JavaScript Object = JavaScript-specific data structure
- Use `JSON.stringify()` to convert Object → JSON
- Use `JSON.parse()` to convert JSON → Object

5. Promise Chaining

- Each `.then()` can return a new Promise
- Chains execute sequentially
- Errors skip to `.catch()`
- `.finally()` always executes

6. Error Handling

- Check `response.ok` for server errors
- Manually `throw` errors for bad responses
- Let `.catch()` handle all errors

7. Advantages Over Callbacks

- Flat structure (no nesting)
 - Better error handling
 - No inversion of control
 - Easier to read and maintain
-

Common Mistakes to Avoid

1. Not Returning Promises in Chains

Wrong:

```
fetch(url)
  .then((response) => {
    response.json(); // Forgot to return!
  })
  .then((data) => {
    console.log(data); // data is undefined!
  });
```

Correct:

```
fetch(url)
  .then((response) => {
    return response.json(); // Return the promise!
  })
  .then((data) => {
    console.log(data); // Now data is available
  });
```

2. Thinking Server Errors = Rejection

Wrong assumption:

```
fetch("https://api.github.com/wrong-url")
  .then((response) => {
    // This executes! (404 is still a response)
  })
  .catch((error) => {
    // This does NOT execute!
  });
```

Correct approach:

```
fetch("https://api.github.com/wrong-url")
  .then((response) => {
    if (!response.ok) {
      throw new Error("Not found");
    }
    return response.json();
  })
  .catch((error) => {
    // Now this catches 404 errors
  });
```

3. Not Handling Errors

Wrong:

```
fetch(url)
  .then((response) => response.json())
  .then((data) => console.log(data));
// No .catch() - errors will be unhandled!
```

Correct:

```
fetch(url)
  .then((response) => response.json())
  .then((data) => console.log(data))
  .catch((error) => console.error(error)); // Always add .catch()
```

4. Reading Promises Directly

Wrong:

```
const p1 = fetch(url);
console.log(p1); // Promise { <pending> } - not useful!
```

Correct:

```
const p1 = fetch(url);
p1.then((response) => {
  console.log(response); // Now you get the actual response
});
```

Practice Exercises

Exercise 1: Basic Promise

Create a Promise that resolves after 2 seconds with a message "Hello, World!":

```
const promise = new Promise((resolve, reject) => {
  setTimeout(() => {
    resolve("Hello, World!");
  }, 2000);
});
```

```
promise.then((message) => {
  console.log(message);
});
```

Exercise 2: Error Handling

Create a Promise that randomly succeeds or fails:

```
const randomPromise = new Promise((resolve, reject) => {
  const random = Math.random();
  setTimeout(() => {
    if (random > 0.5) {
      resolve("Success!");
    } else {
      reject("Failed!");
    }
  }, 1000);
});
```

```
randomPromise
  .then((message) => console.log(message))
  .catch((error) => console.log(error));
```

Exercise 3: Promise Chaining

Create a chain of operations that simulate cooking:

```
const buyIngredients = () => {
  return new Promise((resolve) => {
    setTimeout(() => {
      console.log("Ingredients bought");
      resolve({ ingredients: true });
    }, 1000);
  });
};

const cook = (data) => {
  return new Promise((resolve) => {
    setTimeout(() => {
      console.log("Food cooked");
      data.cooked = true;
      resolve(data);
    }, 1000);
  });
};

const serve = (data) => {
  return new Promise((resolve) => {
    setTimeout(() => {
      console.log("Food served");
      data.served = true;
      resolve(data);
    }, 1000);
  });
};

buyIngredients()
  .then((data) => cook(data))
  .then((data) => serve(data))
  .then((data) => console.log("Complete:", data))
  .catch((error) => console.log("Error:", error))
```

```
.finally(() => console.log("Cleanup done"));
```

Next Steps

In the next lesson, we'll learn about **Async/Await**, which makes working with Promises even easier!

Async/Await provides:

- Cleaner syntax
- Looks like synchronous code
- Easier to read and write
- Better error handling with try/catch

Stay tuned! 🚀

These notes cover everything discussed in the lecture with detailed explanations, examples, and practical implementations. Practice these concepts thoroughly before moving to async/await!